🔷 What are Generic Collections in C#?

Generic collections are **type-safe, reusable data structures** that store elements of a specific type.

👉 Defined in namespace:

using System.Collections.Generic;

✅ Why Generics?

- Type safety (no boxing/unboxing)
- Better performance
- Cleaner code
- Compile-time checking

🔶 Common Interfaces Behind Generic Collections

Before jumping into classes, understand the backbone:

Interface	Purpose
IEnumerable<T>	Enables iteration (foreach)
ICollection<T>	Add, Remove, Count
IList<T>	Index-based access
IDictionary<TKey, TValue>	Key-value pairs
ISet<T>	Unique elements

🔷 1. List

👉 Most commonly used collection (Dynamic Array)

✅ Interfaces Implemented:

- IList<T>
- ICollection<T>
- IEnumerable<T>

📌 **Example:**

```
List<int> numbers = new List<int>();

numbers.Add(10);
numbers.Add(20);
numbers.Add(30);

Console.WriteLine(numbers[1]); // 20
```

✅ **Key Features:**

- Index-based access
- Dynamic resizing
- Allows duplicates

⚡ **Real-time Example:**

```
List<string> students = new List<string>
{
   "Ram", "Shyam", "Krishna"
};
```

🔷 **2. Dictionary<TKey, TValue>**

👉 Stores data as **Key-Value pairs**

✅ **Interfaces Implemented:**

- IDictionary<TKey, TValue>
- ICollection<KeyValuePair<TKey, TValue>>
- IEnumerable<KeyValuePair<TKey, TValue>>

📌 **Example:**

```
Dictionary<int, string> emp = new Dictionary<int, string>();


emp.Add(1, "John");

emp.Add(2, "David");


Console.WriteLine(emp[1]); // John
```

✅ **Key Features:**

- Fast lookup (O(1))
- Unique keys
- No duplicate keys

⚡ **Real-time Example:**

```
Dictionary<string, int> marks = new Dictionary<string, int>

{

   { "Math", 90 },

   { "Science", 85 }

};
```

🔷 **3. HashSet**

👉 Stores **unique elements only**

✅ **Interfaces Implemented:**

- ISet<T>
- ICollection<T>
- IEnumerable<T>

📌 **Example:**

```
HashSet<int> set = new HashSet<int>();


set.Add(1);

set.Add(2);

set.Add(2); // ignored


foreach (var item in set)

{

   Console.WriteLine(item);

}
```

✅ **Key Features:**

- No duplicates
- Fast search
- Unordered

⚡ **Real-time Example:**

```
HashSet<string> emails = new HashSet<string>
{
   "a@gmail.com",
   "b@gmail.com",
   "a@gmail.com" // duplicate ignored
};
```

🔷 **4. LinkedList**

👉 Doubly linked list (nodes connected)

✅ **Interfaces Implemented:**

- ICollection<T>
- IEnumerable<T>

📌 **Example:**

```
LinkedList<string> list = new LinkedList<string>();

list.AddFirst("A");
list.AddLast("B");
list.AddLast("C");

foreach (var item in list)
{
   Console.WriteLine(item);
}
```

✅ **Key Features:**

- Fast insert/delete
- No index access
- Node-based

⚡ **Real-time Example:**

Undo/Redo systems

🔷 **5. Stack**

👉 LIFO (Last In First Out)

✅ **Interfaces Implemented:**

- ICollection<T>
- IEnumerable<T>

📌 **Example:**

```
Stack<int> stack = new Stack<int>();

stack.Push(10);
stack.Push(20);

Console.WriteLine(stack.Pop()); // 20
```

✅ **Key Features:**

- LIFO
- Push, Pop, Peek

⚡ **Real-time Example:**

- Undo operations
- Function calls (call stack)

🔷 **6. Queue**

👉 FIFO (First In First Out)

✅ **Interfaces Implemented:**

- ICollection<T>
- IEnumerable<T>

📌 **Example:**

```
Queue<string> queue = new Queue<string>();


queue.Enqueue("A");

queue.Enqueue("B");


Console.WriteLine(queue.Dequeue()); // A
```

✅ **Key Features:**

- FIFO
- Enqueue, Dequeue

⚡ **Real-time Example:**

- Print queue
- Task scheduling

🔥 **Quick Comparison Table**

Collection	Order	Access	Duplicates	Use Case
List	Ordered	Index	Allowed	General use
Dictionary	Key-based	Fast lookup	Keys unique	Lookup data
HashSet	Unordered	No index	No duplicates	Unique items
LinkedList	Ordered	No index	Allowed	Frequent insert/delete
Stack	LIFO	Top only	Allowed	Undo
Queue	FIFO	Front/Rear	Allowed	Scheduling

🧠 **When to Use What**

- Use **List** → when you need index access
- Use **Dictionary** → when lookup is priority
- Use **HashSet** → when uniqueness matters
- Use **LinkedList** → when frequent insert/delete
- Use **Stack** → LIFO scenarios
- Use **Queue** → FIFO scenarios